EE 542 – Laboratory #2: Fast, Reliable File Transfer

Name: Jacinta Rose Menezes

Name: Yichen Zheng

Name: Junjun Liu

Abstract

In Lab2, we implement a UDP-based file transfer utility designed to achieve fast and reliable transmission under given network conditions. The protocol was built to support large file transfers (≥1 GB) across emulated links with different RTT, packet loss, and throughput constraints.

Our design divides files into MTU-sized chunks, each tagged with sequence numbers, and employs selective retransmission based on negative acknowledgments (NACKs). The client initiates transfer with a metadata handshake (file size, name), streams data chunks rapidly without waiting for per-packet acknowledgments, and signals completion via explicit end-of-transfer packets. The server writes chunks directly to file offsets, detects missing data, and requests only lost packets, minimizing overhead. Reliability is verified by MD5 checksums on both ends.

Key characteristics of our protocol include:

- **High throughput:** using pipelining packets to avoid stop-and-wait inefficiency.
- **Reliability:** ensures correctness through selective retransmission and completion acknowledgments.
- **MTU-awareness:** adapts to both 1500B and jumbo 9000B MTU environments.
- **Robustness:** performs consistently under conditions of delay, loss, and rate limits.

Our approach combines ideas from TCP (sequence tracking), QUIC (UDP-based reliability), and TFTP (simple file transfer semantics), but optimizes for reduced latency and overhead. Experimental results demonstrate that the protocol sustains competitive throughput across all test scenarios, fulfilling the lab's requirement for a reliable, high-performance file transfer utility.

Simulating Networking Environments

Set egress rate limit on both server and client to restrict speed to 100Mbit/s

```
vyos@vyos:~$ sudo tc qdisc add dev eth1 root handle 1:0 tbf rate 100mbit latency
 0.001ms burst 9015
vyos@vyos:~$ sudo tc qdisc add dev eth2 root handle 1:0 tbf rate 100mbit latency
 0.001ms burst 9015
vyos@vyos:~$
```

```
server@server:~$ sudo tc qdisc add dev ens33 root tbf rate 100mbit latency 0.001ms burst 9015
```

```
client@client:~$ sudo tc qdisc add dev ens33 root tbf rate 100mbit latency 0.001ms burst 9015
```

Verification:

```
client@client:~$ iperf3 -c 192.168.10.100 -b 100M
Connecting to host 192.168.10.100, port 5201
[  5] local 192.168.20.100 port 41492 connected to 192.168.10.100 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  11.8 MBytes  98.6 Mbits/sec   11    117 KBytes
[  5]   1.00-2.00   sec  10.9 MBytes  91.2 Mbits/sec    9    109 KBytes
[  5]   2.00-3.00   sec  10.9 MBytes  91.2 Mbits/sec   17   97.6 KBytes
[  5]   3.00-4.00   sec  11.2 MBytes  94.4 Mbits/sec    6    130 KBytes
[  5]   4.00-5.00   sec  10.9 MBytes  91.3 Mbits/sec   10    120 KBytes
[  5]   5.00-6.00   sec  10.9 MBytes  91.2 Mbits/sec    3    110 KBytes
[  5]   6.00-7.00   sec  11.0 MBytes  92.3 Mbits/sec   14    100 KBytes
[  5]   7.00-8.00   sec  11.1 MBytes  93.3 Mbits/sec    9    130 KBytes
[  5]   8.00-9.00   sec  10.9 MBytes  91.2 Mbits/sec   18    120 KBytes
[  5]   9.00-10.00  sec  10.9 MBytes  91.2 Mbits/sec    8    110 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec   110 MBytes  92.6 Mbits/sec  105             sender
[  5]   0.00-10.01  sec   110 MBytes  92.2 Mbits/sec                  receiver

iperf Done.
client@client:~$ iperf3 -u -c 192.168.10.100 -b 200M
Connecting to host 192.168.10.100, port 5201
[  5] local 192.168.20.100 port 60901 connected to 192.168.10.100 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  23.8 MBytes   200 Mbits/sec  17270
[  5]   1.00-2.00   sec  23.8 MBytes   200 Mbits/sec  17245
[  5]   2.00-3.00   sec  23.8 MBytes   200 Mbits/sec  17270
[  5]   3.00-4.00   sec  23.8 MBytes   200 Mbits/sec  17263
[  5]   4.00-5.00   sec  23.9 MBytes   200 Mbits/sec  17279
[  5]   5.00-6.00   sec  23.8 MBytes   200 Mbits/sec  17264
[  5]   6.00-7.00   sec  23.8 MBytes   200 Mbits/sec  17253
[  5]   7.00-8.00   sec  23.8 MBytes   200 Mbits/sec  17271
[  5]   8.00-9.00   sec  23.8 MBytes   200 Mbits/sec  17265
[  5]   9.00-10.00  sec  23.8 MBytes   200 Mbits/sec  17263
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.00  sec   238 MBytes   200 Mbits/sec  0.000 ms  0/172643 (0%)  sender
[  5]   0.00-10.01  sec   115 MBytes  96.5 Mbits/sec  0.131 ms  89268/172636 (52%)  receiver

iperf Done.
```

MTU: 1500

```
vyos@vyos:~$ sudo ip link set dev eth0 mtu 1500
vyos@vyos:~$ sudo ip link set dev eth1 mtu 1500
vyos@vyos:~$ sudo ip link set dev eth2 mtu 1500
vyos@vyos:~$
```

Verification:

```
vyos@vyos:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface    IP Address         MAC                VRF        MTU  S/L    Descri
ption
-----------  -----------------  -----------------  -------  -----  -----  ------
-------
eth0         192.168.184.50/24  00:0c:29:87:25:bd  default   1500  u/u    WAN
             192.168.59.128/24
eth1         192.168.10.1/24    00:0c:29:87:25:c7  default   1500  u/u    LAN-Se
rver
eth2         192.168.20.1/24    00:0c:29:87:25:d1  default   1500  u/u    LAN-Cl
ient
lo           127.0.0.1/8        00:00:00:00:00:00  default  65536  u/u
             ::1/128
```

```
server@server:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc tbf state UP mode DEFAULT group default
    link/ether 00:0c:29:36:69:7d brd ff:ff:ff:ff:ff:ff
    altname enp2s1
```

```
client@client:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc tbf state UP mode DEFAULT group default
    link/ether 00:0c:29:d4:9c:dc brd ff:ff:ff:ff:ff:ff
    altname enp2s1
```

MTU: 9001

```
vyos@vyos:~$ sudo ip link set dev eth2 mtu 9001
vyos@vyos:~$ sudo ip link set dev eth1 mtu 9001
vyos@vyos:~$ sudo ip link set dev eth0 mtu 9001
```

Verification:

```
vyos@vyos:~$ show interfaces
Codes: S - State, L - Link, u - Up, D - Down, A - Admin Down
Interface    IP Address         MAC                VRF        MTU  S/L    Descri
ption
-----------  -----------------  -----------------  -------  -----  -----  ------
-------
eth0         192.168.184.50/24  00:0c:29:87:25:bd  default   9001  u/u    WAN
             192.168.59.128/24
eth1         192.168.10.1/24    00:0c:29:87:25:c7  default   9001  u/u    LAN-Se
rver
eth2         192.168.20.1/24    00:0c:29:87:25:d1  default   9001  u/u    LAN-Cl
ient
lo           127.0.0.1/8        00:00:00:00:00:00  default  65536  u/u
             ::1/128
```

```
client@client:~$ sudo ip link set dev ens33 mtu 9001
client@client:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc fq_codel state UP mode DEFAULT group defa
    link/ether 00:0c:29:d4:9c:dc brd ff:ff:ff:ff:ff:ff
    altname enp2s1
```

```
server@server:~$ sudo ip link set dev ens33 mtu 9001
[sudo] password for server:
server@server:~$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc fq_codel state UP mode DEFAULT group
    link/ether 00:0c:29:36:69:7d brd ff:ff:ff:ff:ff:ff
    altname enp2s1
```

Case 1:

```
vyos@vyos:~$ sudo tc qdisc add dev eth1 parent 1:1 handle 10: netem delay 5ms dr
op 1%
vyos@vyos:~$ sudo tc qdisc add dev eth2 parent 1:1 handle 10: netem delay 5ms dr
op 1%
vyos@vyos:~$
```

Verification:

```
client@client:~$ iperf3 -u -c 192.168.10.100 -b 100M
Connecting to host 192.168.10.100, port 5201
[  5] local 192.168.20.100 port 37714 connected to 192.168.10.100 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  11.9 MBytes   100 Mbits/sec  8630
[  5]   1.00-2.00   sec  11.9 MBytes   100 Mbits/sec  8632
[  5]   2.00-3.00   sec  11.9 MBytes   100 Mbits/sec  8631
[  5]   3.00-4.00   sec  11.9 MBytes   100 Mbits/sec  8634
[  5]   4.00-5.00   sec  11.9 MBytes   100 Mbits/sec  8630
[  5]   5.00-6.00   sec  11.9 MBytes   100 Mbits/sec  8631
[  5]   6.00-7.00   sec  11.9 MBytes   100 Mbits/sec  8636
[  5]   7.00-8.00   sec  11.9 MBytes   100 Mbits/sec  8632
[  5]   8.00-9.00   sec  11.9 MBytes   100 Mbits/sec  8634
[  5]   9.00-10.00  sec  11.9 MBytes   100 Mbits/sec  8630
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.00  sec   119 MBytes   100 Mbits/sec  0.000 ms  0/86320 (0%)  sender
[  5]   0.00-10.12  sec   117 MBytes  96.8 Mbits/sec  0.212 ms  1746/86316 (2%)  receiver

iperf Done.
```

Case 2:

```
vyos@vyos:~$ sudo tc qdisc change dev eth1 parent 1:1 handle 10: netem delay 100
ms drop 20%
vyos@vyos:~$ sudo tc qdisc change dev eth2 parent 1:1 handle 10: netem delay 100
ms drop 20%
vyos@vyos:~$ _
```

Verification:

```
client@client:~$ iperf3 -u -c 192.168.10.100 -b 100M
Connecting to host 192.168.10.100, port 5201
[  5] local 192.168.20.100 port 40258 connected to 192.168.10.100 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  11.9 MBytes  99.9 Mbits/sec  8627
[  5]   1.00-2.00   sec  11.9 MBytes   100 Mbits/sec  8634
[  5]   2.00-3.00   sec  11.9 MBytes   100 Mbits/sec  8633
[  5]   3.00-4.00   sec  11.9 MBytes   100 Mbits/sec  8631
[  5]   4.00-5.00   sec  11.9 MBytes   100 Mbits/sec  8635
[  5]   5.00-6.00   sec  11.9 MBytes   100 Mbits/sec  8631
[  5]   6.00-7.00   sec  11.9 MBytes   100 Mbits/sec  8630
[  5]   7.00-8.00   sec  11.9 MBytes   100 Mbits/sec  8633
[  5]   8.00-9.00   sec  11.9 MBytes   100 Mbits/sec  8634
[  5]   9.00-10.00  sec  11.9 MBytes   100 Mbits/sec  8633
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.00  sec   119 MBytes   100 Mbits/sec  0.000 ms  0/86321 (0%)  sender
[  5]   0.00-10.21  sec  88.1 MBytes  72.4 Mbits/sec  0.138 ms  22550/86321 (26%)  receiver

iperf Done.
```

Case 3:

```
vyos@vyos:~$ sudo tc qdisc del dev eth0 root
vyos@vyos:~$ sudo tc qdisc del dev eth1 root
vyos@vyos:~$ sudo tc qdisc del dev eth2 root
vyos@vyos:~$ sudo tc qdisc add dev eth0 root tbf rate 100mbit latency 0.001ms bu
rst 9015
vyos@vyos:~$ sudo tc qdisc add dev eth1 root handle 1:0 tbf rate 80mbit latency
0.001ms burst 9015
vyos@vyos:~$ sudo tc qdisc add dev eth2 root handle 1:0 tbf rate 80mbit latency
0.001ms burst 9015
vyos@vyos:~$ sudo tc qdisc add dev eth1 parent 1:1 handle 10: netem delay 100ms
vyos@vyos:~$ sudo tc qdisc add dev eth2 parent 1:1 handle 10: netem delay 100ms
vyos@vyos:~$ _
```

Verification:

```
client@client:~$ iperf3 -u -c 192.168.10.100 -b 100M
Connecting to host 192.168.10.100, port 5201
[  5] local 192.168.20.100 port 54216 connected to 192.168.10.100 port 5201
[ ID] Interval           Transfer     Bitrate         Total Datagrams
[  5]   0.00-1.00   sec  11.9 MBytes  99.9 Mbits/sec  8629
[  5]   1.00-2.00   sec  11.9 MBytes  99.9 Mbits/sec  8630
[  5]   2.00-3.00   sec  11.9 MBytes   100 Mbits/sec  8632
[  5]   3.00-4.00   sec  11.9 MBytes   100 Mbits/sec  8635
[  5]   4.00-5.00   sec  11.9 MBytes   100 Mbits/sec  8633
[  5]   5.00-6.00   sec  11.9 MBytes   100 Mbits/sec  8631
[  5]   6.00-7.00   sec  11.9 MBytes   100 Mbits/sec  8633
[  5]   7.00-8.01   sec  11.9 MBytes  99.2 Mbits/sec  8630
[  5]   8.01-9.00   sec  11.9 MBytes   101 Mbits/sec  8636
[  5]   9.00-10.00  sec  11.9 MBytes   100 Mbits/sec  8630
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Jitter    Lost/Total Datagrams
[  5]   0.00-10.00  sec   119 MBytes   100 Mbits/sec  0.000 ms  0/86319 (0%)  sender
[  5]   0.00-12.50  sec  86.5 MBytes  58.0 Mbits/sec  2.072 ms  23694/86314 (27%)  receiver

iperf Done.
client@client:~$ iperf3  -c 192.168.10.100 -t 10
Connecting to host 192.168.10.100, port 5201
[  5] local 192.168.20.100 port 38316 connected to 192.168.10.100 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  1.25 MBytes  10.5 Mbits/sec    0    226 KBytes
[  5]   1.00-2.00   sec  3.88 MBytes  32.5 Mbits/sec    0    451 KBytes
[  5]   2.00-3.00   sec  2.38 MBytes  19.9 Mbits/sec    0    509 KBytes
[  5]   3.00-4.00   sec  2.38 MBytes  19.9 Mbits/sec    0    549 KBytes
[  5]   4.00-5.00   sec  2.38 MBytes  19.9 Mbits/sec    0    574 KBytes
[  5]   5.00-6.00   sec  3.50 MBytes  29.4 Mbits/sec    0    587 KBytes
[  5]   6.00-7.00   sec  2.50 MBytes  21.0 Mbits/sec    0    592 KBytes
[  5]   7.00-8.00   sec  2.25 MBytes  18.9 Mbits/sec    0    592 KBytes
[  5]   8.00-9.00   sec  3.62 MBytes  30.4 Mbits/sec    0    592 KBytes
[  5]   9.00-10.00  sec  2.38 MBytes  19.9 Mbits/sec    0    595 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  26.5 MBytes  22.2 Mbits/sec    0             sender
[  5]   0.00-10.20  sec  24.6 MBytes  20.2 Mbits/sec                  receiver

iperf Done.
```

Concept & Explanation of File Transfer Utility

Our file transfer utility is a reliable data transfer protocol based on UDP because UDP is lightweight, connectionless, and fast, which fits the lab requirement the most. However, UDP does not guarantee reliability, so we design a reliable layer including:

- **Handshake** to exchange file metadata and confirm readiness.
- **Chunk-based transfer** to divide a file into MTU-sized packets with sequence numbers.
- **Selective retransmission** triggered by **NACKs (Negative Acknowledgments)** from the server.
- **Final acknowledgment** to confirm the transfer's successful completion.

We borrow the idea from QUIC (Quick UDP Internet Connections). It is a modern transport protocol developed by Google, now standardized by the IETF (RFC 9000).

QUIC is also based on UDP, and it introduces a **Low-Latency** Handshake before transfer to ensure reliability:
1-RTT: New connections complete the handshake in just one round trip.
0-RTT: If a client has connected before, it can start sending data immediately, even before the handshake is complete.

In our protocol, we perform the handshake as:
The client sends **FILE_INFO** (file size, name, chunk count).
The server replies with **ACK_DONE** to confirm readiness.

When executing the data transfer, we split the file into **MTU-sized chunks** with sequence numbers and the chunks are sent rapidly, without waiting for per-packet acknowledgments. It is similar to QUIC, which also assigns packet sequence numbers and can send them out of order, with reassembly handled by the receiver. In this way, the sender can send many packets at the same time, which gives a higher throughput and better bandwidth utilization. And our protocol only sends

NACKs when something is missing, which gives lower feedback overhead, making sure less bandwidth is wasted on acknowledgments.

Workflow

a) Handshake

1. The client sends FILE_INFO, which consists of file size, name, and total packet count.
2. Server pre-allocates storage space and replies with ACK_DONE.
3. This ensures both sides agree on transfer parameters before sending data.

b) Data Transfer

1. Client splits the file into chunks of size
2. Each chunk is wrapped in a FILE_DATA packet with its sequence number.
3. Packets are sent rapidly, without waiting for per-packet acknowledgments (unlike stop-and-wait protocols).
4. The server writes data directly at the proper file offset using the sequence number.

c) End of Bulk Transfer

1. After sending all chunks, the client transmits FILE_END packets (repeated a few times for reliability).
2. The server detects this marker and transitions into the repair phase.

d) Repair (NACK Phase)

1. Server checks for missing chunks by comparing received sequence numbers with the expected total.
2. Missing numbers are grouped and sent back to the client in NACK packets.

3. Client retransmits only the requested chunks (Selective Repeat mechanism).
4. This loop continues until no missing chunks remain or a retry limit is reached.

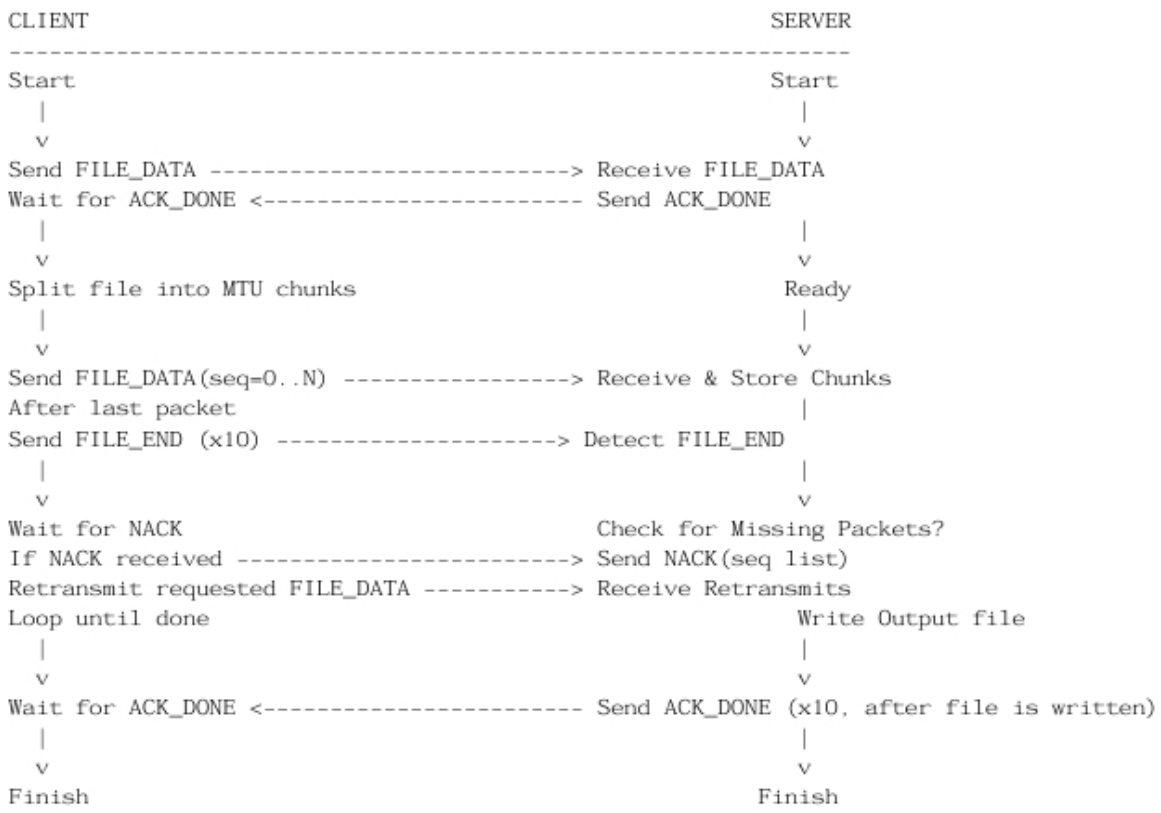
e) Completion

1. Once the server verifies file completeness, it sends multiple ACK_DONE messages.
2. Client, upon receiving ACK_DONE, terminates the session.
3. Both sides close the sockets and finalize the file transfer.

Core Characteristics

- **Reliability:** Achieved via selective retransmission using NACKs.
- **Efficiency:** No per-packet ACKs → reduces feedback traffic, unlike TCP.
- **Ordering:** Maintained by sequence numbers; server reassembles file at correct offsets.
- **Flow Control:** Implicit — sender blasts packets but stops/resends only when asked.
- **Loss Recovery:** Server-driven (NACK-based) instead of sender-driven timeout.
- **Robustness:** Control packets (FILE_END, ACK_DONE) are sent multiple times to overcome potential losses.
- **Simplicity:** Limited state, smaller implementation compared to full TCP.

Flow Chart of Protocol Used (Algorithm)



Data Structures Used in Code

1. Server:

- **Protocol Headers (PacketHdr, FilePayloadInfo)**: Packed structs carry packet metadata and file information, ensuring consistent byte layout across systems.
- **Packet Tracking (std::vector<bool> received)**: A compact bitmap to record whether each file chunk has been received.
- **Synchronization (std::mutex file_mutex)**: Protects shared resources like the tracking vector and file writes in concurrent threads.
- **File Handling (std::fstream out)**: Used to write chunks to the correct file offset.
- **Buffers (char buffer[65536] & std::vector<char> payload):** Raw arrays handle incoming packets, while dynamic vectors manage variable-size retransmission payloads.

2. Client:

- **Protocol headers (PacketHdr, FilePayloadInfo)**: Packed structs defining control information; PacketHdr stores type, seq, size, and totalpkts, while FilePayloadInfo contains filesize and path.
- **File chunk storage (vector<string> chunks)**: Holds file data split into MTU-sized segments, allowing direct access by sequence number for sending and retransmission.
- **Packet assembly buffer (vector<char> packvec)**: Combines PacketHdr with chunk payloads into a single contiguous memory block for sendto.
- **Thread synchronization (mutex send_mutex)**: Ensures serialized access to the UDP socket when multiple threads call sendto.
- **Temporary receive buffer (char buffer[65536])**: Raw array used for incoming messages such as ACK_DONE and NACK.

Results

1. MTU - 1500:

- **Case 1 (RTT=10ms, Loss rate=1%, Throughput=100Mbits/s)**
  The client is sending the 1GB file (bigfile.bin) to the server.
  Total time taken is 87s.

```
client@client:~$ ./client ./bigfile.bin
[CLIENT] Using MTU payload size: 1484
[CLIENT] Handshake done
[CLIENT] Sent FILE_END marker
[CLIENT] Total time: 87.0759 seconds
```

  The server is outputting the received packets into the output file (outputfile1.bin)

```
server@server:~$ ./server ./outputfile1.bin
[SERVER] Using MTU payload size: 1484
[SERVER] Listening on port 9000
[SERVER] FILE_INFO received: ./outputfile1.bin size=1073741824 totalpkts=723546
[SERVER] FILE_END received
[SERVER] Total time: 87.0637 seconds
```

  Verification of the file:

```
server@server:~$ md5sum ./outputfile1.bin
cd573cfaace07e7949bc0c46028904ff  ./outpu
client@client:~$ md5sum ./bigfile.bin
cd573cfaace07e7949bc0c46028904ff  ./bi
```

- **Case 2 (RTT=200ms, Loss rate=20%, Throughput=100Mbits/s)**
  The client is sending the 1GB file (bigfile.bin) to the server.
  Total time taken is 59s.

```
client@client:~$ ./client ./bigfile.bin
[CLIENT] Using MTU payload size: 1484
[CLIENT] Handshake done
[CLIENT] Sent FILE_END marker
[CLIENT] Total time: 59.1264 seconds
```

The server is outputting the received packets into the output file (outputfile2.bin)

```
server@server:~$ ./server ./outputfile2.bin
[SERVER] Using MTU payload size: 1484
[SERVER] Listening on port 9000
[SERVER] FILE_INFO received: ./outputfile2.bin size=1073741824 totalpkts=723546
[SERVER] FILE_INFO received: ./outputfile2.bin size=1073741824 totalpkts=723546
[SERVER] FILE_END received
[SERVER] Total time: 58.925 seconds
```

Verification of the file:

```
server@server:~$ md5sum ./outputfile2.bin
cd573cfaace07e7949bc0c46028904ff  ./outpu
client@client:~$ md5sum ./bigfile.bin
cd573cfaace07e7949bc0c46028904ff  /bi
```

- **Case 3 (RTT=200ms, Loss rate=0%, Throughput=80Mbits/s)**
  The client is sending the 1GB file (bigfile.bin) to the server.
  Total time taken is 71s.

```
client@client:~$ ./client ./bigfile.bin
[CLIENT] Using MTU payload size: 1484
[CLIENT] Handshake done
[CLIENT] Sent FILE_END marker
[CLIENT] Total time: 71.7414 seconds
```

The server is outputting the received packets into the output file (outputfile2.bin)

```
server@server:~$ ./server ./outputfile3.bin
[SERVER] Using MTU payload size: 1484
[SERVER] Listening on port 9000
[SERVER] FILE_INFO received: ./outputfile3.bin size=1073741824 totalpkts=723546
[SERVER] FILE_END received
[SERVER] Total time: 71.2555 seconds
```

Verification of the file:

```
server@server:~$ md5sum ./outputfile3.bin
cd573cfaace07e7949bc0c46028904ff  ./outputfile3.bin
client@client:~$ md5sum ./bigfile.bin
cd573cfaace07e7949bc0c46028904ff  /bi
```

2. MTU - 9001:

- **Case 1 (RTT=10ms, Loss rate=1%, Throughput=100Mbits/s)**
  The client is sending the 1GB file (bigfile.bin) to the server.
  Total time taken is 25s.

```
client@client:~$ ./client ./bigfile.bin
[CLIENT] Using MTU payload size: 8973
[CLIENT] Handshake done
[CLIENT] Sent FILE_END marker
[CLIENT] Total time: 25.3611 seconds
```

  The server is outputting the received packets into the output file (outputfile4.bin)

```
server@server:~$ ./server ./outputfile4.bin
[SERVER] Using MTU payload size: 8973
[SERVER] Listening on port 9000
[SERVER] FILE_INFO received: ./outputfile4.bin size=1073741824 totalpkts=119664
[SERVER] FILE_END received
[SERVER] Total time: 24.7463 seconds
```

  Verification of the file:

```
server@server:~$ md5sum ./outputfile4.bin
cd573cfaace07e7949bc0c46028904ff  ./outputfile4.bin
```

```
client@client:~$ md5sum ./bigfile.bin
cd573cfaace07e7949bc0c46028904ff  /bi
```

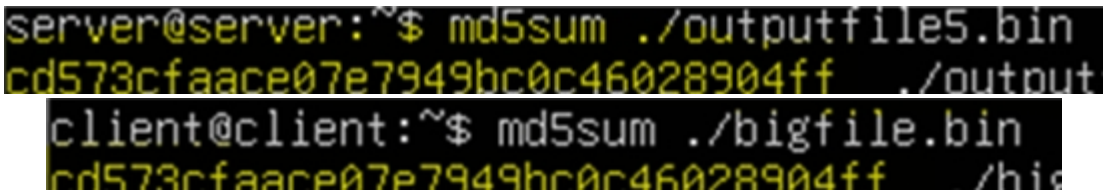
- **Case 2 (RTT=200ms, Loss rate=20%, Throughput=100Mbits/s)**
  The client is sending the 1GB file (bigfile.bin) to the server.
  Total time taken is 25s.

```
client@client:~$ ./client ./bigfile.bin
[CLIENT] Using MTU payload size: 8973
[CLIENT] Handshake done
[CLIENT] Sent FILE_END marker
[CLIENT] Total time: 25.7444 seconds
```

  The server is outputting the received packets into the output file (outputfile5.bin)

```
server@server:~$ ./server ./outputfile5.bin
[SERVER] Using MTU payload size: 8973
[SERVER] Listening on port 9000
[SERVER] FILE_INFO received: ./outputfile5.bin size=1073741824 totalpkts=119664
[SERVER] FILE_END received
[SERVER] Total time: 25.3131 seconds
```

Verification of the file:



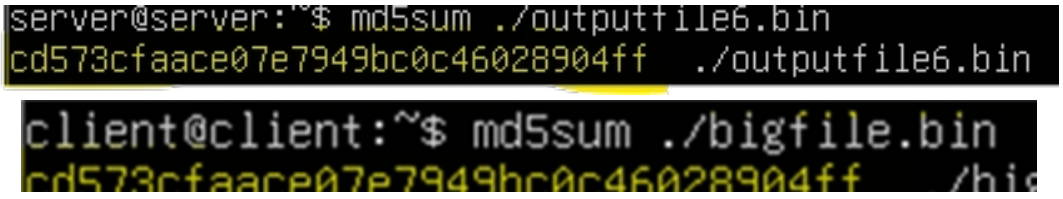
- **Case 3 (RTT=200ms, Loss rate=0%, Throughput=80Mbits/s)**
  The client is sending the 1GB file (bigfile.bin) to the server.
  Total time taken is 25s.

```
client@client:~$ ./client ./bigfile.bin
[CLIENT] Using MTU payload size: 8973
[CLIENT] Handshake done
[CLIENT] Sent FILE_END marker
[CLIENT] Total time: 25.6748 seconds
```

The server is outputting the received packets into the output file (outputfile6.bin)

```
server@server:~$ ./server ./outputfile6.bin
[SERVER] Using MTU payload size: 8973
[SERVER] Listening on port 9000
[SERVER] FILE_INFO received: ./outputfile6.bin size=1073741824 totalpkts=119664
[SERVER] FILE_END received
[SERVER] Total time: 25.4623 seconds
```

Verification of the file:



Analysis

1. Compare Table:

Metric	MTU1500	MTU 9001
Avg. Transfer Time	~72s	25s
Variability Across Runs	High (59–87s)	Low (all 25s)
Reliability	Correct, with retries	Correct, minimal retries
Efficiency	Moderate, high overhead	High, reduced packetization cost

2. Observation:

MTU Impact on Performance

For MTU = 1500, transfer times varied significantly between runs because the system must process a very large number of small packets. With the high counts of packets, even low loss rate result in frequent retransmission, and the overhead grows disproportionately as the number of missing chunks increases, leading to unstable completion times as our protocol uses NACK-based recovery.

For MTU = 9001, results were both faster and consistent. The transfer time consistently 25s across all cases. The reduction in the number of packets per file

transfer lowered the protocol overhead and minimized retransmission events. As a result, throughput scaled close to the physical rate limit.

Case Comparisons

The three test cases show how **latency, loss,** and **bandwidth** limits interact with protocol design:

- **Case 1 (RTT 10ms, 1% loss):**
  With small latency and low loss, both MTU 1500 and 9001 sustained almost maximum throughput. However, MTU 1500 still suffered longer completion time (~87s worst-case) due to the high number of retransmissions triggered by even 1% packet drops. MTU 9001 completed in 25s reliably.

- **Case 2 (RTT 200ms, 20% loss):**
  The harshest condition. Despite 20% loss, our NACK-based protocol maintained **59–71s completion with MTU 1500** and **25s with MTU 9001**. Unlike TCP—the client could continue sending packets, while the server efficiently requested only missing chunks. Still, MTU 1500 showed instability because of repeated retransmission loops, while MTU 9001 remained robust.

- **Case 3 (RTT 200ms, no loss, 80 Mbps):**
  The throughput was limited by the router's rate control. MTU 1500 achieved ~71s, while MTU 9001 still reached ~25s. The absence of packet loss means retransmissions barely happened, but packetization overhead with MTU 1500 remained costly.

3. Key Findings:

Reliability: The NACK + Selective Retransmission mechanism ensured complete and correct file reconstruction under both MTU conditions.

Efficiency: Jumbo Frames improved throughput by more than 3x, compared with the best MTU 1500 case.

Stability: Results under MTU 9001 were consistent, while MTU 1500 showed performance fluctuations due to retransmission sensitivity.

Protocol Design Impact: The protocol's lightweight acknowledgment strategy (NACK-based rather than per-packet ACKs) contributed to efficiency gains, but its effectiveness scaled much better with larger MTU sizes.

Conclusion

Our lab demonstrates a UDP-based file transfer protocol that can provide both high throughput and reliability for large-scale data transmission. By introducing a metadata handshake (inspired by QUIC), MTU-aware segmentation, and pipelined delivery of chunks, our protocol avoids the inefficiencies of stop-and-wait schemes. Reliability is ensured through NACK-driven selective retransmission and redundant control signaling, while ordering is maintained via sequence numbers and direct offset writes. Together, these designs enable the system to achieve correctness with minimal acknowledgment overhead.

Experimental evaluation confirms that the protocol is robust across different MTU settings and network conditions. Transfers with the standard 1500B MTU remain reliable but show variability in completion time due to retransmission sensitivity. In contrast, jumbo MTU (9000B) significantly reduces packetization overhead, achieving stable performance and up to a threefold increase in throughput.

Overall, the results highlight the effectiveness of combining simplicity with targeted reliability mechanisms for a fast, reliable, and efficient file transfer utility.